

# Laser Distance Meter X-40 / Hi-AT Technology

UART protocol documentation draft for building an Arduino/ESP32 library

Version: 0.1 draft - generated 2026-05-16 - language: German

Ziel: Dieses Dokument soll direkt an Codex/Claude/Gemini gegeben werden koennen, damit daraus eine kleine, robuste Library entsteht. Es dokumentiert die wahrscheinlich passende ASCII-UART-Protokollvariante fuer den X-40/40m-Laser-Distanzmesser und markiert unsichere Punkte explizit.

Wichtig: Ich habe kein eindeutig offizielles Datenblatt exakt fuer 'Hi-AT X-40' gefunden. Die wichtigsten Protokolldaten sind aus sehr aehnlichen 40m/50m/100m Laser-Distance-Modulen und Arduino-Testberichten abgeleitet. Vor der finalen Library sollten die Befehle auf dem konkreten Geraet am Tisch verifiziert werden.

## 1. Quellenlage und Vertrauensstufen

Die folgenden Angaben sind als technische Arbeitsgrundlage gedacht, nicht als Herstellerfreigabe. Fuer die Library sollen daher Timeouts, Rohantworten und unbekannte Fehlercodes sauber nach aussen gegeben werden.

| Aussage                                             | Vertrauen      | Begrueendung                                                                                                   |
|-----------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------|
| UART TTL, 19200 Baud, 8N1, keine Paritaet           | hoch           | Mehrere 40m/50m/100m Module listen diese Schnittstelle und dieselben Kommandos.                                |
| Befehle O/C/D/S als einzelne ASCII-Zeichen          | hoch           | Mehrere Arduino-Quellen und Haendlertexte nennen genau diese Kommandos.                                        |
| Antwortformat Distanz: z.B. 12.345m,0079            | hoch           | Arduino-Testcode und Modulbeschreibungen verwenden dieses Format.                                              |
| SQ nach Komma = Signalqualitaet; kleiner ist besser | mittel-hoch    | In Arduino-Testberichten so beschrieben; sinnvoll fuer Reflexionsqualitaet.                                    |
| Befehle M/F/V/X                                     | mittel/niedrig | Bei verwandten Hi-AT/JRT-Familien plausibel, aber fuer X-40 nicht sicher. Als optionale Erweiterung behandeln. |
| Versorgung 2.8 V bzw. 2.0-3.3 V                     | mittel         | Aehnliche Module variieren. Nicht ungeprueft 5 V anlegen.                                                      |

## 2. Elektrische Schnittstelle

Die Ziel-Library soll die elektrische Verdrahtung nicht erzwingen, aber im README deutlich warnen. Besonders bei 5V-Boards ist ein Pegelwandler oder mindestens ein geeigneter Spannungsteiler auf der Leitung vom MCU-TX zum Modul-RX noetig.

| Signal am Modul | Richtung | Zum Controller                           | Hinweis                                                                             |
|-----------------|----------|------------------------------------------|-------------------------------------------------------------------------------------|
| VCC             | Input    | 2.8 V bevorzugt, je nach Modul 2.0-3.3 V | Nicht blind 5 V. Stromreserve ca. 150-300 mA vorsehen.                              |
| GND             | -        | GND                                      | Gemeinsame Masse zwingend.                                                          |
| TX              | Output   | MCU RX                                   | TTL/CMOS-Pegel. Bei ESP32 direkt meist passend.                                     |
| RX              | Input    | MCU TX                                   | Bei 5V-MCU Pegel reduzieren. Bei ESP32 3.3 V meist passend.                         |
| Power-EN / EN   | Input    | optional GPIO                            | Falls herausgefuehrt: Enable/Power-Steuerung. Nicht bei jedem Handgeraet vorhanden. |

Empfohlene Startkonfiguration fuer ESP32: HardwareSerial, 19200 Baud, SERIAL\_8N1, RX/TX frei wahlbar. Serielle Ausgabe des Moduls kann CR, LF, CRLF oder nur nach kurzer Idle-Zeit enden; deshalb den Parser nicht nur auf ein einzelnes Endzeichen verlassen.

## 3. UART-Grundeinstellungen

| Parameter    | Wert fuer ASCII-Protokoll |
|--------------|---------------------------|
| Baudrate     | 19200                     |
| Datenbits    | 8                         |
| Paritaet     | keine                     |
| Stopbits     | 1                         |
| Flow Control | keine                     |

| Parameter    | Wert fuer ASCII-Protokoll                                      |
|--------------|----------------------------------------------------------------|
| Frame-Art    | ASCII, meist ein einzelnes Grossbuchstaben-Kommando ohne CR/LF |
| Antwort-Ende | CR/LF, oder Timeout/Idle-Gap robust behandeln                  |

## 4. Befehle

Die Library soll mindestens die Befehle O, C, D und S implementieren. Optionale Befehle duerfen existieren, muessen aber als 'unsupported' oder Timeout erkannt werden, falls das konkrete X-40 nicht antwortet.

| Befehl | Hex  | Name                    | Erwartete Antwort                  | Library-Methode                      |
|--------|------|-------------------------|------------------------------------|--------------------------------------|
| O      | 0x4F | Laser einschalten       | OK, ,OK! oder Text mit OK          | laserOn()                            |
| C      | 0x43 | Laser ausschalten       | OK, ,OK! oder Text mit OK          | laserOff()                           |
| S      | 0x53 | Status lesen            | z.B. 18.0C, 2.7V oder 18.0'C, 3.0V | readStatus()                         |
| D      | 0x44 | Distanz messen          | z.B. 12.345m,0079                  | measure(),<br>readRangeMillimeters() |
| M      | 0x4D | langsame/genaue Messung | Distanzformat oder Timeout         | measurePrecise() optional            |
| F      | 0x46 | schnelle Messung        | Distanzformat oder Timeout         | measureFast() optional               |
| V      | 0x56 | Version/Info            | freier Text oder Timeout           | readVersion() optional               |
| X      | 0x58 | Modul ausschalten       | keine oder OK                      | powerDown() optional                 |

## 5. Antwortformate und Parsing

Der Parser soll nie voraussetzen, dass die Antwort exakt gleich lang ist. Fuehrende Leerzeichen, Komma-Abstand und Grad-/Apostroph-Zeichen im Status sollen toleriert werden.

### 5.1 Distanzantwort

Beispiele:

```
12.345m,0079
 2.345m, 0079
0.532m,0123
Er.XX!
```

Regel:

- Suche das erste 'm' oder 'M'.
- Lies davor die letzte zusammenhaengende Zahl als Meterwert.
- Suche nach Komma; danach die erste Ganzzahl als Signalqualitaet SQ.
- Millimeter =  $\text{round}(\text{meter} * 1000.0)$ .
- Falls keine Zahl oder 'Er.' erkannt wird: Fehlerstatus zurueckgeben.

Empfohlene Struktur fuer die Library:

```
struct X40Measurement {
    bool ok;
    float meters;
    int32_t millimeters;
    int signalQuality;    // -1 falls nicht vorhanden
    int errorCode;        // -1 falls kein Er.xx-Code
    String raw;           // immer fuer Debugging behalten
};
```

### 5.2 Statusantwort

Beispiele:

```
18.0C, 2.7V
18.0'C, 3.0V
```

Regel:

- Erste Float-Zahl vor C als Temperatur in deg C.
- Erste Float-Zahl vor V als Versorgungsspannung.
- Wenn nur Rohtext kommt: raw behalten, valid=false.

## 6. Fehlerbehandlung

Antworten der Form Er.xx oder Er.XX! muessen als Geraetefehler behandelt werden. Ohne offizielles X-40-Manual sollte die Library den numerischen Code nicht zu selbstsicher uebersetzen. Die README kann allgemeine Ursachen nennen: Ziel ausser Reichweite, zu schwaches/starkes Signal, starke Fremdbeleuchtung, Unterspannung, Temperatur ausserhalb

Bereich, Hardwarefehler.

| Situation                        | Library-Verhalten                                     |
|----------------------------------|-------------------------------------------------------|
| Timeout                          | ok=false, status=Timeout, raw leer oder Teilantwort   |
| Antwort enthaelt Er.             | ok=false, status=DeviceError, errorCode extrahieren   |
| Antwort enthaelt keine Meterzahl | ok=false, status=ParseError, raw behalten             |
| OK erwartet, aber anderer Text   | ok=false oder status=UnexpectedResponse; raw behalten |
| SQ fehlt                         | Messwert darf trotzdem ok sein, signalQuality=-1      |

## 7. Vorgeschlagene Library-API

Die API soll moeglichst nahe an einer typischen Ranging-Sensor-Library liegen: begin(), available/status, single measurement, readRangeMillimeters(), Timeout-Steuerung. Falls du schon eine bestehende Library-API hast, soll Codex die Namen entsprechend angleichen, aber die folgenden Semantiken beibehalten.

```
class X40LaserDistanceMeter {
public:
    enum Status {
        OK = 0,
        Timeout,
        ParseError,
        DeviceError,
        UnexpectedResponse,
        Unsupported
    };

    bool begin(Stream& serial);
    bool begin(HardwareSerial& serial, int rxPin, int txPin,
               uint32_t baud = 19200);

    void setTimeout(uint32_t ms);           // default 4000 ms fuer Messung
    void setIdleGap(uint32_t ms);          // default 30-50 ms nach letztem Byte

    bool laserOn();
    bool laserOff();
    bool readStatus(float* temperatureC, float* supplyV);
    bool readVersion(String* versionText); // optional

    X40Measurement measure();              // default: 'D'
    X40Measurement measurePrecise();        // optional: 'M'
    X40Measurement measureFast();          // optional: 'F'

    int32_t readRangeMillimeters();         // kompatibler Convenience-Call
    float readRangeMeters();               // NAN bei Fehler

    Status lastStatus() const;
    int lastErrorCode() const;
    const String& lastRawResponse() const;
};
```

## 8. Empfohlene Implementierungsdetails

| Thema                 | Empfehlung                                                                                       |
|-----------------------|--------------------------------------------------------------------------------------------------|
| Serial flush          | Vor jedem Kommando alle alten Bytes lesen/verwerfen.                                             |
| Senden                | Nur ein Byte schreiben, z.B. serial.write('D'); kein CR/LF erzwingen.                            |
| Lesen                 | Bis CR/LF oder bis idleGap nach letztem Byte oder bis total timeout.                             |
| Blocking              | Messbefehle duerfen blocken, aber nur bis timeout. Keine endlosen while-Schleifen.               |
| Rohdaten              | Jede Antwort im Objekt speichern und optional an Debug-Callback geben.                           |
| Autodetect            | Optional: S senden. Wenn keine Antwort, Baud 115200 nur als Diagnose testen, nicht als Standard. |
| Laser-Lifetime        | Nach Einzelmessung optional laserOff(), aber nicht automatisch wenn Nutzer continuous/scan will. |
| String vs char buffer | Auf kleinen MCUs lieber festen char buffer; ESP32 kann String tolerieren.                        |

## 9. Minimaler Testablauf fuer Codex

1. Hardware verbinden: GND-GND, Modul TX -> MCU RX, Modul RX -> MCU TX, VCC passend versorgen (nicht blind 5 V).

2. Serial auf 19200 8N1 starten.
3. Befehl 'S' senden. Erwartung: Temperatur/Spannung oder wenigstens Rohantwort.
4. Befehl 'O' senden. Erwartung: OK.
5. Befehl 'D' senden. Erwartung: Meterwert, z.B. 1.234m,00xx.
6. Befehl 'C' senden. Erwartung: OK.
7. Danach M/F/V/X nur als optionale Feature-Tests ausfuehren.

Wenn Schritt 3-5 nicht funktioniert: RX/TX tauschen, Versorgung messen, Pegel pruefen, 1 kOhm bis 4.7 kOhm Serienwiderstand oder Spannungsteiler in MCU-TX -> Modul-RX, und auf echte 19200 Baud pruefen. Falls weiterhin keine Antwort kommt, kann das konkrete Handgeraet intern eine andere Protokollvariante oder gar keinen herausgefuehrten TTL-Port haben.

## 10. Beispiel fuer Arduino/ESP32-Testsketch

```
#include <Arduino.h>

static const int LDM_RX = 16; // ESP32 RX  <- Modul TX
static const int LDM_TX = 17; // ESP32 TX  -> Modul RX
HardwareSerial LDM(2);

String readResponse(uint32_t timeoutMs = 4000, uint32_t idleGapMs = 40) {
    String s;
    uint32_t t0 = millis();
    uint32_t lastByte = millis();
    bool got = false;

    while (millis() - t0 < timeoutMs) {
        while (LDM.available()) {
            char c = (char)LDM.read();
            if (c >= 32 || c == '\r' || c == '\n') s += c;
            got = true;
            lastByte = millis();
            if (c == '\r' || c == '\n') { s.trim(); return s; }
        }
        if (got && millis() - lastByte >= idleGapMs) break;
        delay(1);
    }
    s.trim();
    return s;
}

String sendCmd(char cmd, uint32_t timeoutMs = 4000) {
    while (LDM.available()) LDM.read();
    LDM.write((uint8_t)cmd);
    return readResponse(timeoutMs);
}

float parseMeters(const String& raw) {
    int mpos = raw.indexOf('m');
    if (mpos < 0) mpos = raw.indexOf('M');
    if (mpos < 0) return NAN;

    int start = mpos;
    while (start > 0) {
        char c = raw[start - 1];
        if (isDigit(c) || c == '.' || c == '-' || c == ' ') start--;
        else break;
    }
    String n = raw.substring(start, mpos);
    n.trim();
    return n.toFloat();
}

int parseSQ(const String& raw) {
    int comma = raw.indexOf(',');
    if (comma < 0) return -1;
    String q = raw.substring(comma + 1);
    q.trim();
    return q.toInt();
}

void setup() {
    Serial.begin(115200);
    LDM.begin(19200, SERIAL_8N1, LDM_RX, LDM_TX);
    delay(300);

    Serial.println("S: " + sendCmd('S', 1000));
    Serial.println("O: " + sendCmd('O', 1000));
}
```

```

void loop() {
  String raw = sendCmd('D', 4000);
  Serial.println("RAW: " + raw);

  if (raw.indexOf("Er.") >= 0) {
    Serial.println("Device error");
  } else {
    float m = parseMeters(raw);
    int sq = parseSQ(raw);
    Serial.printf("m=%.3f mm=%ld SQ=%d\n", m, lroundf(m * 1000.0f), sq);
  }
  delay(1000);
}

```

## 11. Codex-Auftrag: erwartete Dateien

Wenn dieses PDF an Codex gegeben wird, soll der Auftrag ungefaehr so lauten: 'Erstelle eine Arduino-Library fuer den Laser Distance Meter X-40 mit derselben oeffentlichen API wie meine bestehende Ranging-Sensor-Library. Verwende dieses Protokolldokument als Quelle. Implementiere robuste Timeouts, Rohantworten und Parser-Tests.'

| Datei                                           | Inhalt                                                                       |
|-------------------------------------------------|------------------------------------------------------------------------------|
| src/X40LaserDistanceMeter.h                     | Klasse, Status-Enum, Result-Struct, oeffentliche API                         |
| src/X40LaserDistanceMeter.cpp                   | Serial-Kommunikation, Parser, Timeout/Idle-Gap                               |
| examples/BasicRead/BasicRead.ino                | Einzelmessung mit D, Ausgabe in mm/m/SQ                                      |
| examples/StatusAndLaser/StatusAndLaser.ino      | S, O, D, C Sequenz                                                           |
| test/parser_tests.cpp oder docs/parser_tests.md | Beispielformate und erwartete Parse-Ergebnisse                               |
| README.md                                       | Warnungen zu Spannung/Pegel, Verdrahtung, Protokoll, bekannte Unsicherheiten |

## 12. Parser-Testfaelle

| Raw          | Erwartung                                   |
|--------------|---------------------------------------------|
| 12.345m,0079 | ok, meters=12.345, millimeters=12345, SQ=79 |
| 2.345m, 0079 | ok, meters=2.345, millimeters=2345, SQ=79   |
| 0.532m,0123  | ok, meters=0.532, millimeters=532, SQ=123   |
| Er.01!       | DeviceError, errorCode=1                    |
| 18.0C, 2.7V  | Status: temp=18.0, V=2.7                    |
| 18.0°C, 3.0V | Status: temp=18.0, V=3.0                    |
|              | Timeout oder ParseError, je nach Kontext    |

## 13. Abgrenzung zu anderen Protokollen

Einige Laser-Distanzmodule verwenden ein anderes, gerahmtes Kommandoformat wie ASCII-Strings mit '\$...&' oder Modbus/CRC. Dieses Dokument beschreibt die einfache Single-Letter-ASCII-Variante. Die Library sollte nicht versuchen, mehrere Protokolle zu vermischen, ausser Codex implementiert bewusst eine separate Auto-Detect-Schicht.

## 14. Quellen / Links fuer Codex

Die Quellen sind nicht alle gleich stark. Besonders die Arduino-Forum-Beitraege sind praktische Erfahrungswerte, aber keine Herstellerfreigaben. Trotzdem sind sie fuer die UART-Kommandos sehr nuetzlich.

| Quelle                                                                          | Wichtige Information                                                                                                                                  |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arduino Forum: How to use laser sensor 40 m with Arduino                        | Beispielcode mit VCC 2.8 V, 19200 Baud, Befehlen O/C/S/D und Antwort 12.345m,0079.                                                                    |
| Arduino Forum: How to Use Laser Distance Meter / laser Rangefinder With Arduino | Haendlerdaten: TTL, 19.2K, 8N1, keine Paritaet, O/C/D/S, Range/Accuracy eines aehnlichen Moduls.                                                      |
| JRT 703A 40m UART Laser Distance Module                                         | Aehnliches 40m-Modul: 0.03-40m, TTL/CMOS, 2.0-3.3V, 635nm <1mW, RXTX optional.                                                                        |
| J5A30 Ranging Module Datasheet                                                  | Nicht X-40, aber Beispiel fuer anderes UART-TTL-Lasermodule: Pins RX/TX/EN, 3.3-5V, Default 115200. Dient nur als Warnung, dass Protokolle variieren. |

<https://forum.arduino.cc/t/how-to-use-laser-sensor-40-m-with-arduino/464389>

<https://forum.arduino.cc/t/how-to-use-laser-distance-meter-laser-rangefinder-with-arduino/584132>

<https://www.jrt-measure.com/industrial-laser-distance-sensor/57158505.html>

<https://19981102.s21i.faiusr.com/61/5/ABUIABA9GAAG36r9tAYorNWK7QU.pdf>

## 15. Offene Punkte vor finaler Library-Freigabe

| Punkt                              | Wie klaeren                                                                                          |
|------------------------------------|------------------------------------------------------------------------------------------------------|
| Antwort-Endzeichen                 | Log mit Hexdump aufnehmen: CR, LF, CRLF oder nur Idle-Gap.                                           |
| Exakte Versorgung                  | Am Modul/Handgeraet messen oder Datenblatt/PCB-Aufdruck pruefen.                                     |
| M/F/V/X Unterstuetzung             | Jeden Befehl einzeln testen und Timeout protokollieren.                                              |
| Fehlercode-Mapping                 | Absichtlich erst generisch lassen, spaeter aus Manual oder Tests ergaenzen.                          |
| Gleiche API wie bestehende Library | Codex soll die Namen aus der vorhandenen Library uebernehmen und intern auf dieses Protokoll mappen. |